Risk Register

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Risk Register

A UP project is **risk-driven**: the riskiest, least-understood parts are tackled first (Elaboration), not left until the end. This register is a *living* document — it started at Inception and was updated at the end of each later phase, which is what the marking guide's "Updated risk list" (Elaboration 1) and "risk mitigation progress" (Elaboration 2) line items ask for. Rather than three separate snapshots, each risk below carries its status **at every phase it was reassessed**, so the progression is visible in one place.

Likelihood / Impact scale: Low / Medium / High. **Status:** Open → Mitigating → Mitigated → Closed (resolved or accepted).

1. Risk list

ID	Risk	Category	Likelihood	Impact	Mitigation
R1	The client-server layering rule (client never talks to Supabase; routes → services → repositories only) erodes over time as features are added under time pressure	Architectural	Medium	High	Enforced structurally, not just by convention: only `repositories/*.js` import the Supabase client; services take no `req` / `res`; every new feature reviewed against this before merging. See LOGICAL_ARCHITECTURE.md §2.
R2	Hostel Wi-Fi is unreliable; a student mid-submission loses connectivity and their report is lost	Technical / NFR	High	Medium	PWA offline drafting: a new request is written to IndexedDB and synced automatically on reconnect (`draftQueue.js`, `draftSync.js`), rather than failing outright.
R3	Supabase service-role key or other secrets leak into the client bundle or source control	Security	Low	High	The key is used **only** in `repositories/*.js`; `.env` is git-ignored with `.env.example` committed instead; the client build is grepped for the key pattern as a sanity check before each deploy.
R4	A migration is written but never actually run against the live Supabase project, silently breaking a feature	Operational	High	Medium	Migrations `013` onward are written **safe to re-run** (`create table if not exists`); the scheduler logs a distinct, actionable warning (e.g. "has sql/015 been run?") the first time a query fails because a table is missing, so the symptom points straight at the cause.

ID	Risk	Category	Likelihood	Impact	Mitigation
R5	Two students upvote, or two administrators assign, the same request at the same instant, corrupting the count/assignment	Technical	Low	Medium	Database-level guarantees, not just application checks: `request_upvotes` has a **composite primary key** `(request_id, user_id)`, so a duplicate vote is impossible even if the service-layer check were bypassed; assignment is a single transaction-equivalent repository call.
R6	The free hosting tier (Render) sleeps when idle, making the first request after idle very slow, which looks like a broken app to a first-time user	Operational / NFR	High	Low	A scheduled external health-check ping (UptimeRobot → `/api/health` every 5–10 min) keeps the instance warm — see PHYSICAL_ARCHITECTURE.md §1.
R7	The AI triage feature becomes a hard dependency — if the Anthropic API is slow, unavailable or the key is unset, request submission breaks	Technical	Medium	Medium	AI triage is advisory-only, on a separate rate-limited endpoint the submission flow does not require; a deterministic keyword-rules engine answers when the API is unavailable, with an identical response shape so the client does not need to branch on which one answered.
R8	In-memory state (live-update connections, rate limiters, profile cache) does not survive a restart or scale to more than one server instance	Architectural	Medium	Low (at this scale)	Accepted for this deployment's scale and documented explicitly rather than hidden — see PHYSICAL_ARCHITECTURE.md §3 and ARCHITECTURE.md §13/§14. A shared store (Redis) is named as the concrete next step if this ever needs to change.
R9	A student's identity leaks through the "already reported" community feed, discouraging honest reporting	Privacy	Low	High	The feed is server-assembled with **no name or id field in its response shape** at all — not merely hidden in the UI — so there is nothing to leak even if a client render bug exposed every field it received; covered by an integration test asserting no identity fields are present.
R10	Feature scope grows faster than the 15-week timeline can absorb (11 features were added in a single late-session request at one point)	Schedule	High	Medium	Strict "commit after each feature" discipline kept every addition independently reviewable, testable and revertible, so scope growth added incremental risk per feature rather than compounding risk across a big-bang batch.

ID	Risk	Category	Likelihood	Impact	Mitigation
R11	The integration test suite fakes the database, so a real PostgreSQL/RLS bug could pass CI and only surface in production	Quality	Medium	Medium	Explicitly documented as a known limitation rather than an assumed non-issue (TESTING.md §7); every migration is additionally exercised manually against the real Supabase project before being relied on, and RLS policies are reviewed by hand against the same ownership rules the service layer enforces.
R12	A generic placeholder like `<hostel>` written directly in Markdown prose is silently stripped by both GitHub's renderer and the PDF pipeline, since both treat it as an unknown HTML tag	Documentation quality	Medium	Low	Found during a documentation-accuracy pass (see DEFECT_LOG.md, DEF-07); fixed by HTML-escaping (`<hostel>`) everywhere it occurs outside a code span, and now checked for systematically before each doc regeneration.

2. Status by phase

ID	@ Inception	@ Elaboration 1	@ Elaboration 2	@ Construction/Transition (current)
R1	Open — named as the central architectural bet	Mitigating — layering proven on the core lifecycle	Mitigated — held across every later feature (announcements, preventive maintenance, feedback all respected it)	**Closed** — structurally enforced (see R1's mitigation)
R2	Open — identified from the brief's offline-tolerance requirement	Open — not yet built	Mitigating — draft queue designed, not yet implemented	**Mitigated** — implemented, manually tested offline/online
R3	Open	Mitigating — `.env` pattern established	Mitigated	**Closed** — no incidents; verified before every deploy
R4	Not yet identified (no migrations existed yet)	Open — first migration-order issue hit during manual testing	Mitigating — safe-to-re-run pattern adopted from migration 013	**Mitigated** — adopted for all migrations 013–016
R5	Not yet identified	Open — upvotes not yet built	Mitigating — composite PK designed in, migration 009	**Closed** — verified by dedicated integration tests
R6	Not yet relevant (not deployed)	Not yet relevant	Open — noticed once deployed to Render's free tier	**Mitigated** — UptimeRobot configured

ID	@ Inception	@ Elaboration 1	@ Elaboration 2	@ Construction/Transition (current)
R7	Not yet identified	Open — AI triage not yet built	Mitigating — rules-engine fallback designed alongside the AI path from the start	**Closed** — both paths tested, safety floor applies to either
R8	Not yet relevant	Open — SSE/live updates not yet built	Open — accepted trade-off explicitly discussed	**Accepted** — documented as a known limitation, not silently ignored
R9	Open — privacy named as a non-functional requirement	Mitigating — community feed designed with identities excluded from the response	Mitigated	**Closed** — covered by an integration test
R10	Open — flagged given the solo-developer, iterative-request pattern	Mitigating — small-commit discipline established	Mitigated	**Mitigated** — held through the largest single batch (11 features, one late-session request)
R11	Not yet relevant (no tests existed)	Open — test strategy chosen (fake DB for speed and CI-without-secrets)	Open — trade-off confirmed deliberate	**Accepted** — documented in TESTING.md §7 as an honest limitation
R12	Not yet identified	Not yet identified	Not yet identified	**Closed** — found and fixed during the documentation-accuracy pass; see DEFECT_LOG.md DEF-07

Reading this table: every risk that could be engineered away (R1–R7, R9, R10, R12) reached **Closed** or **Mitigated**. The two that remain **Accepted** (R8, R11) are architectural trade-offs made deliberately for this project's scale, not oversights — both are named explicitly in ARCHITECTURE.md's "Known limitations" and "Future work" sections rather than glossed over, which is itself the honest, defensible answer to "what would you build next?" in a viva.